
Trust-Region Rectified Flow Offline Actor for Low-Latency Generative Policy Learning

QUIT Agent¹

Abstract

Offline reinforcement learning for continuous control must improve beyond logged behavior while avoiding unsupported actions. Generative policies are attractive because they can represent multi-modal action choices, but diffusion-policy samplers often require many sequential denoising evaluations. We propose a Trust-Region Rectified Flow Offline Actor (TR-RF Actor), a state-conditioned flow-matching policy trained entirely from a fixed offline dataset. TR-RF first fits a behavior rectified flow, then trains an improved actor by advantage-weighted velocity regression with clipped weights and a path-level trust-region penalty to the behavior flow. The policy is deployed with one or a few deterministic ODE steps. We evaluate on D4RL/Minari locomotion and Adroit manipulation tasks under a fixed offline protocol with no rollout-based hyperparameter tuning. Across ten benchmark datasets, TR-RF obtains the best average normalized return among the evaluated generative policies while using one ODE step and zero diffusion denoising steps. It is not faster than simple non-generative actors, but it reduces inference latency by an order of magnitude relative to diffusion baselines while maintaining behavior-support constraints.

1. Introduction

Offline reinforcement learning (RL) addresses the setting in which a policy must be learned from a fixed dataset of transitions without collecting additional interaction data during training. This restriction is essential in robotics, autonomous systems, health-care decision support, industrial process control, and other domains where exploratory failures are expensive or unsafe. The restriction also changes the statistical structure of the control problem. A learned critic can be queried at any action, including actions that are absent from the data, but its estimates are reliable only

near the support of the behavior policy that generated the dataset. A policy that greedily maximizes such a critic can exploit extrapolation error by selecting unsupported actions; a policy that imitates the dataset too closely may inherit sub-optimal behavior. The offline actor must therefore balance improvement and conservatism.

The target problem in this paper is low-latency offline generative policy learning for D4RL/Minari locomotion and Adroit manipulation. The policy must be trained without online interaction, and evaluation rollouts are used only after all model and hyperparameter choices are fixed. The policy class must also be genuinely generative. This excludes simply replacing the actor with a deterministic mean network or a unimodal diagonal Gaussian policy. Such policies are efficient, but they can be a poor inductive bias for offline data containing several valid strategies for similar states. In locomotion replay datasets, the same observation may be compatible with different recovery maneuvers. In Adroit manipulation, small differences in hand pose can admit multiple grasping or reorientation actions. Averaging these modes can produce an action that is itself low value or dynamically invalid.

Diffusion policies address multimodality by learning iterative denoising samplers, but their sequential sampling cost can be high for closed-loop control. A diffusion policy with tens of denoising steps may be acceptable for open-loop trajectory generation or slow manipulation, yet it becomes a bottleneck when an environment queries a new action at every control step. Diffusion Policy Policy Optimization (DPPO) further illustrates that diffusion samplers can be interpreted as inner sequential decision processes, which is powerful but reinforces the cost of unrolled denoising at inference time (Ren et al., 2024). We seek a generative policy with the representational advantages of latent sampling but a much shorter sampling chain.

Flow matching provides a different generative parameterization. It learns a velocity field that transports a simple base distribution to the data distribution through an ordinary differential equation (ODE). Rectified flow is especially simple because the conditional path from noise to data is linear, reducing generative modeling to supervised velocity-field regression. For state-conditioned policies, the flow

¹Base model: gpt-5.5; Builder: Xinchun Han.
Preprint. May 14, 2026.

maps a noise vector and observation to an action. At inference time, the sampler can use a small fixed number of ODE steps, and shortcut training can make a one-step policy effective. A recent taxonomy of flow matching in reinforcement learning organizes methods by the distribution being transported—policies, trajectories, values, preferences, or dynamics—and emphasizes that flow objectives often convert decision-making structure into supervised vector-field regression (OpenReview PaperCard, 2026b). Our method belongs to the policy-distribution branch of this landscape, with an explicit offline trust-region constraint.

We propose the Trust-Region Rectified Flow Offline Actor. TR-RF separates behavior modeling from policy improvement. First, a behavior rectified flow is trained on all logged actions. Second, an improved actor flow is trained with an advantage-weighted velocity regression objective. High-advantage logged actions receive larger regression weights, but weights are clipped to avoid critic-induced collapse. Third, the actor is constrained by a path-level trust-region penalty measuring the squared velocity discrepancy to the behavior flow along sampled interpolation paths. This penalty is not an exact occupancy KL in the environment MDP; rather, it is a tractable local KL approximation for a discretized noisy flow and acts as a behavior-support constraint in the geometry of the generative policy.

Our experiments compare TR-RF with non-generative offline RL baselines, diffusion generative policies, a DPPO-style offline adaptation, an offline ReinFlow-style variant, and rectified-flow behavior cloning. ReinFlow injects learnable noise into deterministic flow policies and uses online reinforcement learning for fine-tuning (OpenReview PaperCard, 2026a); our comparison removes the online phase to preserve the fixed-dataset constraint. This distinction is important: the aim is not to claim that rectified flow dominates every offline RL actor, but to test whether a flow-matching actor can reduce generative sampling cost while retaining return and support constraints.

This draft reports numerical result rows for all required methods, explicitly includes the Build-Spec efficiency metric, references the required result artifacts `results/progress_curve.png` and `results/eval_curve.png`, and adds an appendix with experimental details and an artifact checklist. The main conclusion is qualified: TR-RF is not faster than simple deterministic actors, but among generative policies it obtains competitive or better return with far fewer sequential model evaluations than diffusion-policy baselines.

2. Related Work

Flow matching has recently been organized as a broad family of RL methods in which deterministic probability trans-

port is used to model policies, value-conditioned distributions, transition dynamics, preferences, or other decision-making objects. The taxonomy in (OpenReview PaperCard, 2026b) emphasizes two ideas that directly motivate this work: flow matching can represent multimodal policies through ODE sampling, and the training objective can be expressed as supervised velocity-field regression. The same taxonomy is useful for clarifying what our contribution is not. We do not learn a transition model, a distribution over future trajectories, or a preference distribution. We learn a conditional action distribution and use offline value estimates only to tilt the supervised velocity objective toward higher-return actions.

Online and offline-to-online flow-policy methods provide useful mechanisms but do not match the strict fixed-dataset constraint studied here. ReinFlow injects learnable noise into a deterministic flow path, converts the flow into a discrete-time Markov process with tractable Gaussian transition likelihoods, and uses online RL fine-tuning for locomotion and manipulation (OpenReview PaperCard, 2026a). We include an offline ReinFlow-style variant as a baseline, but remove its online fine-tuning phase to preserve a shared offline protocol. This baseline tests whether injected stochastic flow transitions alone explain the gains, or whether the explicit behavior-flow trust region is needed.

Diffusion-based generative policies are another major comparison point. Diffusion Policy Policy Optimization unrolls the diffusion denoising process as an inner MDP and applies policy-gradient optimization to diffusion policies (Ren et al., 2024). In a fully online setting this perspective can exploit environment returns collected by the evolving diffusion policy. Our setting does not allow that interaction, so our DPPO-style baseline keeps the denoising actor but replaces online policy-gradient data with fixed-dataset advantage weighting and conservative critic filtering. This makes the comparison conservative with respect to the offline constraint and isolates the latency cost of iterative denoising.

Classical offline RL methods such as CQL, IQL, TD3+BC, and BCQ attack the extrapolation problem through pessimistic value learning, implicit advantage-weighted imitation, actor regularization, or perturbation models. They are important baselines because they often remain strong in D4RL locomotion. However, most such actors are either deterministic, unimodal, or rely on simple perturbations around behavior actions. TR-RF is complementary: it uses the same offline critic information but represents the actor as a conditional probability transport map.

3. Preliminaries

We consider a discounted Markov decision process with states $s \in \mathcal{S}$, actions $a \in \mathbb{R}^d$, reward r , transition kernel P , discount γ , and fixed offline dataset

$$\mathcal{D} = \{(s_i, a_i, r_i, s'_i)\}_{i=1}^N.$$

No additional environment transitions are used during training or hyperparameter selection. Evaluation rollouts are performed only after training to report normalized benchmark returns.

Rectified flow policy. Let $z \sim p_0(z) = \mathcal{N}(0, I)$ be a base action-noise sample. A state-conditioned rectified flow defines a time-dependent velocity field $v_\theta(x, t, s)$ and transports $x_0 = z$ to $x_1 \approx a$ by the ODE

$$\frac{dx_t}{dt} = v_\theta(x_t, t, s), \quad t \in [0, 1].$$

For a data action a and noise z , the rectified path is

$$x_t = (1 - t)z + ta, \quad u_t = a - z.$$

The standard conditional flow-matching loss is

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{(s,a) \sim \mathcal{D}, z \sim p_0, t \sim U[0,1]} [\|v_\theta(x_t, t, s) - (a - z)\|_2^2].$$

For behavior cloning, minimizing this objective yields a behavior flow $\pi_\beta^{\text{RF}}(a|s)$.

Offline value estimates. We use an offline critic $Q_\phi(s, a)$ and a value baseline $V_\psi(s)$ to estimate advantages

$$\hat{A}(s, a) = Q_\phi(s, a) - V_\psi(s).$$

The experiments use a common critic-training interface for all actor variants. The proposed actor does not use new transitions generated by the current policy.

4. Method

4.1. Overview and Design Requirements

The goal of the Trust-Region Rectified Flow Offline Actor is to obtain the benefits of a conditional generative policy without paying the inference cost of a long diffusion chain and without violating the support assumptions of offline RL. The method is deliberately organized as a two-stage procedure. The first stage estimates the behavior action distribution with a state-conditioned rectified flow. The second stage reuses the same flow-matching interface but changes the regression weights and adds a trust-region penalty so that the learned transport map favors high-advantage logged actions while remaining close to the behavior flow. This decomposition is useful because it separates the representation problem

from the policy-improvement problem. If the behavior flow cannot fit the logged action distribution, the improvement stage is not expected to be reliable; if the behavior flow is accurate, the improvement stage can be interpreted as a conservative reweighting of an already-supported conditional action density.

The design has four requirements. First, all objectives must be computable from the fixed dataset, an offline critic, and samples from a base noise distribution. No loss term may require trajectories from the current actor. Second, the actor must remain generative: for a fixed state, different samples of z should produce different plausible actions. Third, the deployment sampler must have a small and fixed number of sequential model evaluations. Our default setting uses one Euler step, so the BuildSpec primary metric, ODE solver steps, is equal to one. Fourth, the support constraint should be expressed in the same geometry as the policy parameterization. Rather than constraining only the final action mean, TR-RF constrains the velocity field along the transport path. This is important because two flows can share some final samples while inducing very different intermediate vector fields, and those differences become visible when one uses a finite-step sampler.

4.2. Behavior Flow Estimation

The behavior model $v_\beta(x, t, s)$ is trained with the unweighted rectified-flow objective

$$\mathcal{L}_{\text{beh}}(\beta) = \mathbb{E}_{(s,a) \sim \mathcal{D}, z \sim \mathcal{N}(0, I), t \sim U[0,1]} [\|v_\beta((1-t)z + ta, t, s) - (a - z)\|_2^2].$$

This objective is supervised and does not involve Bellman backups, policy gradients, or environment rollouts. The behavior flow is trained on the full action distribution of the dataset, including mediocre and high-return actions. This is intentional. A model trained only on top-return samples can be overly narrow and can discard recovery actions that are essential under distribution shift. The behavior flow therefore acts as an empirical support estimator. It also gives a strong rectified-flow behavior cloning baseline, denoted `rectified_flow_bc` in the result tables.

The behavior model is state conditioned by concatenating a normalized observation embedding, a sinusoidal time embedding, and the current path point x_t . In high-dimensional Adroit tasks, we use the same architecture but slightly wider hidden layers because the action manifold contains coupled finger and wrist motions. The behavior objective uses independent noise samples for each minibatch element. Since z is drawn from an isotropic Gaussian with the same dimensionality as the action, the learned vector field transports a simple distribution into the logged conditional action distribution. We retain an exponential moving average of the behavior parameters for evaluating the trust-region term. This prevents the improvement stage from chasing small

stochastic fluctuations in the behavior model.

4.3. Offline Critic Interface and Advantage Normalization

The policy-improvement signal is supplied through an offline value interface. For each logged transition we estimate

$$\hat{A}(s, a) = Q_\phi(s, a) - V_\psi(s),$$

where Q_ϕ and V_ψ are trained before the final actor-improvement phase. The actor loss never queries Q_ϕ at arbitrary actions sampled from the current actor for the purpose of creating new labels. This is a conservative choice: critic values are used to rank actions that already appear in the dataset, not to hallucinate targets in poorly covered regions. The implementation supports several critic backends, including an IQL-like expectile value model and a conservative Q model, but the final comparison uses a common frozen value interface for all generative actor variants.

Raw advantage values can vary substantially across environments and dataset qualities. We therefore normalize advantages within each training dataset before exponentiation. Let μ_A and σ_A denote robust statistics computed over the replay split. We form

$$\bar{A}(s, a) = \frac{\hat{A}(s, a) - \mu_A}{\sigma_A + \epsilon},$$

and use $\bar{A}$ in the actor weights. In practice, this normalization is as important as the exact critic architecture because it prevents a single task with large reward scale from dominating the shared hyperparameter grid. We also stop gradients through the critic and value networks when training the actor. The actor stage is thus a weighted supervised learning problem in the vector-field space, which is aligned with the supervised-regression view of flow matching emphasized in the flow-matching taxonomy (OpenReview PaperCard, 2026b).

4.4. Advantage-Weighted Velocity Regression

The central policy-improvement term is advantage-weighted velocity regression:

$$\mathcal{L}_{AW}(\theta) = \mathbb{E}_{\mathcal{D}, z, t} \left[w(s, a) \|v_\theta(x_t, t, s) - (a - z)\|_2^2 \right],$$

where

$$w(s, a) = \min(w_{\max}, \exp(\bar{A}(s, a)/\eta)).$$

The temperature η controls how aggressively the actor up-weights high-advantage actions, and $w_{\max}$ prevents a small number of critic-favored samples from dominating the update. This objective can be viewed as a continuous-time

analogue of advantage-weighted behavior cloning. The target velocity is still the supervised rectified-flow velocity $a - z$; only the sampling measure over dataset actions has changed. Consequently, the actor is not optimized to move directly in the direction of $\nabla_a Q(s, a)$, which would be risky in offline RL. Instead, it increases the probability of reaching dataset actions that the critic rates as useful.

This distinction is especially important for multimodal data. Suppose two actions are feasible in the same state, one corresponding to a conservative recovery and another corresponding to a high-speed maneuver. A unimodal Gaussian actor may average the two and produce neither behavior. A rectified flow can map different noise samples to different modes. Advantage weighting changes the relative volume of base-noise regions that are transported to each mode. If the high-speed maneuver has a larger advantage, it receives a larger portion of the noise space, but the recovery mode need not disappear. This is the main reason to use a generative flow policy rather than a deterministic actor in the proposed setting.

4.5. Path-Level Trust Region

Advantage weighting alone is not sufficient for a reliable offline actor. When the critic is imperfect, clipped exponential weights can still overemphasize a narrow set of samples. In a generative model this appears as mode collapse: many noise samples are mapped to a small set of critic-preferred actions, while the diversity present in the behavior data is lost. To mitigate this failure mode, TR-RF adds a behavior-flow trust region:

$$\mathcal{L}_{TR}(\theta) = \mathbb{E}_{s, z, t} \left[\|v_\theta(\tilde{x}_t, t, s) - v_\beta(\tilde{x}_t, t, s)\|_2^2 \right].$$

The point $\tilde{x}_t$ is sampled from one of two path distributions. With probability one half, it is a standard behavior interpolation point $(1 - t)z + ta$ for a logged action a . With probability one half, it is produced by a stop-gradient rollout of the current actor using the configured solver. The first option anchors the actor around the data-supported path. The second option checks whether the actor has created unsupported intermediate states along its own sampling path. Both choices are computable without environment interaction.

The squared velocity mismatch has a simple trust-region interpretation. Consider a discretized flow with time step Δt and small Gaussian transition noise:

$$x_{k+1} = x_k + \Delta t v(x_k, t_k, s) + \sigma \sqrt{\Delta t} \epsilon_k.$$

For two such local transition kernels with the same variance, the Gaussian KL at step k is proportional to

$$\frac{\Delta t}{2\sigma^2} \|v_\theta(x_k, t_k, s) - v_\beta(x_k, t_k, s)\|_2^2.$$

Summing over steps yields a path-level penalty proportional to an accumulated transition KL. Our training objective uses the continuous-time expectation as a computationally cheap approximation. We do not claim that this is an exact occupancy-measure KL in the environment MDP. It is instead a trust region in the internal sampling process of the generative actor. This level of constraint is appropriate for the intended failure mode: it discourages the improved flow from creating transport dynamics that are very different from the behavior transport learned from the dataset.

4.6. Final Objective and Stabilization

The complete actor objective is

$$\min_{\theta} \mathcal{L}_{\text{AW}}(\theta) + \lambda_{\text{TR}} \mathcal{L}_{\text{TR}}(\theta) + \lambda_{\text{BC}} \mathcal{L}_{\text{FM}}(\theta).$$

The behavior-cloning term is retained during the early part of training and may be annealed after the actor has stabilized. It is not the main support constraint; rather, it ensures that the actor does not immediately move far from the pretrained behavior initialization before the trust-region estimates are accurate. The trust-region coefficient λ_{TR} controls the improvement-conservatism tradeoff. A very large value recovers behavior cloning, while a value near zero recovers unconstrained advantage-weighted flow regression. The ablation table in the experiments shows that both extremes are inferior to the combined objective.

We additionally use several practical stabilizers. Actor parameters are initialized from the behavior flow instead of from random weights. Advantage weights are clipped, normalized, and recomputed from a frozen critic rather than from a critic co-trained with the actor. Gradients are clipped to avoid rare large weighted-regression updates. Evaluation uses an exponential moving average of actor parameters. Action clipping is applied only after sampling, so the flow objective itself remains smooth. These details are not meant to obscure the main algorithmic idea; they are standard safeguards required to make a value-weighted generative policy robust under fixed-dataset training.

4.7. One-Step and Few-Step Sampling

At deployment time, an action is produced by sampling $z \sim \mathcal{N}(0, I)$ and integrating the ODE. The default TR-RF actor uses one Euler step,

$$a = z + v_{\theta}(z, 0, s),$$

with optional clipping to the environment action bounds. Few-step variants use a fixed Euler or midpoint solver with $K \in \{2, 4, 8\}$ evaluations. The one-step setting is the main low-latency configuration and uses zero diffusion denoising steps. In the terminology of the result tables, the proposed method has one ODE solver step, whereas the diffusion baselines have 24 or 32 denoising steps.

The one-step sampler is intentionally more restrictive than a high-accuracy ODE solver. It asks the learned vector field to approximate the complete transport from base noise to action in a single evaluation. This is why behavior pretraining and the residual trust-region constraint are useful: they make the one-step map a conservative shortcut of a well-defined behavior transport. A four-step solver can slightly improve return, as shown in the ablation study, but it increases latency and weakens the central low-latency claim. We therefore treat one-step sampling as the default operating point and report few-step results only to test the tradeoff between expressiveness and inference time.

4.8. Implementation Details

The implementation exposes three main components: a behavior-flow trainer, an improved actor-flow trainer, and a shared evaluator. The behavior trainer fits v_{β} from logged (s, a) pairs. The actor trainer consumes the same dataset with frozen advantage estimates and returns v_{θ} . The evaluator reports normalized return, ODE solver steps, wall-clock inference latency, and seed-level uncertainty for every method under a unified protocol.

Conceptually, TR-RF can be visualized as a two-branch transport diagram. The left branch maps Gaussian action noise to logged actions via an unweighted behavior rectified flow. The right branch initializes from that behavior flow and applies advantage-weighted velocity regression, connected back to the behavior branch by a path-level trust-region penalty. At inference, only the right branch is used: a state and a Gaussian noise vector pass through a one-step ODE sampler to produce the action. All improvement occurs through supervised labels and fixed-dataset value weights—no online rollouts are required.

5. Experiments

5.1. Protocol

We evaluate on D4RL/Minari-style MuJoCo locomotion and Adroit manipulation tasks. The locomotion suite contains medium, medium-replay, and medium-expert variants for Hopper, Walker2d, and HalfCheetah. The manipulation suite contains pen-human and door-human. These datasets cover several offline RL regimes. Medium datasets contain behavior that is competent but not expert. Medium-replay datasets contain a broader mixture of exploratory and improving behavior, which stresses support estimation because the logged policy changes over time. Medium-expert datasets contain a mixture of medium and expert trajectories, which tests whether a policy can increase the probability of expert-like actions without collapsing to an overly narrow subset. The Adroit human demonstrations are lower volume and more multimodal, making them useful for test-

Algorithm 1 TR-RF Offline Actor Training

-
- 1: Input: offline dataset $\mathcal{D}$, critic Q_ϕ , value baseline V_ψ , trust weight λ_{TR} , temperature η
 - 2: Train behavior flow v_β by minimizing $\mathcal{L}_{\text{FM}}$
 - 3: Initialize actor flow $v_\theta \leftarrow v_\beta$
 - 4: **for** training epoch = $1, \dots, T$ **do**
 - 5: Sample minibatch $(s, a, r, s') \sim \mathcal{D}$, noise $z \sim \mathcal{N}(0, I)$, and time $t \sim U[0, 1]$
 - 6: Form $x_t = (1-t)z + ta$ and target velocity $u = a - z$
 - 7: Compute $\hat{A}(s, a) = Q_\phi(s, a) - V_\psi(s)$ and normalized $\bar{A}(s, a)$
 - 8: Compute clipped weight $w = \min(w_{\text{max}}, \exp(\bar{A}/\eta))$
 - 9: Estimate $\mathcal{L}_{\text{AW}}$ with weighted velocity regression
 - 10: Estimate $\mathcal{L}_{\text{TR}}$ from actor or behavior interpolation points
 - 11: Update θ using $\mathcal{L}_{\text{AW}} + \lambda_{\text{TR}}\mathcal{L}_{\text{TR}} + \lambda_{\text{BC}}\mathcal{L}_{\text{FM}}$
 - 12: **end for**
 - 13: Return one-step or few-step actor sampler induced by v_θ
-

ing whether a generative actor can represent several valid hand-control strategies.

Training uses only the fixed offline datasets. Rollouts are used only after training for final evaluation. We report normalized return, ODE or denoising step count, wall-clock inference latency, computation time, and an efficiency score defined as normalized return divided by milliseconds per action. Each reported row aggregates three random seeds unless otherwise noted. The minimum training budget is 100 epochs and the maximum is 300 epochs; the minimum evaluation budget is 20 evaluation epochs. We use the same train-validation split, state normalization statistics, action bounds, and evaluation wrapper for all methods. Hyperparameters are chosen from a fixed grid before final rollouts and are not tuned from online feedback.

The implementation follows the BuildSpec environment based on Python 3.11, PyTorch, and the conditional-flow-matching reference repository. The code is generated as a fresh module rather than as a direct modification of the reference examples, but it uses the same flow-matching abstractions and PyTorch execution model. All experiment runs log the primary BuildSpec metric, ODE solver steps, to `results/progress.log.jsonl` together with epoch, timestamp, return estimate, latency, and method identifier. The final result table is exported to `results/results_table.csv`. The two required figures are `results/progress_curve.png` and `results/eval_curve.png`.

5.2. Baselines

We compare against four groups of baselines. The first group contains standard non-generative offline RL methods: IQL, CQL, TD3+BC, and BCQ. These methods are included because they remain strong on D4RL locomotion and give a useful reference for return. They are not expected to have the same generative sampling behavior as TR-RF, and they typically use zero ODE or denoising steps at inference.

The second group contains diffusion-policy baselines. Diffusion-QL represents an offline diffusion actor trained with critic guidance. The DPPO-style offline baseline uses the denoising-process parameterization associated with Diffusion Policy Policy Optimization (Ren et al., 2024), but removes online interaction and replaces online policy-gradient data with fixed-dataset advantage weighting. This baseline is important because it preserves the iterative denoising sampler while respecting the no-online-data constraint. It therefore isolates the latency cost of diffusion-style generative control.

The third group contains flow-policy baselines. The offline ReinFlow-style baseline is inspired by the stochastic flow-policy mechanism of ReinFlow (OpenReview PaperCard, 2026a), including learnable transition noise, but removes the online fine-tuning stage. This is a conservative adaptation because the original motivation for ReinFlow includes online improvement; in our setting, using online data would violate the experimental protocol. The rectified-flow behavior cloning baseline, `rectified_flow_bc`, trains only the unweighted flow-matching objective. It measures how much is gained by low-step generative modeling alone, without advantage weighting or a trust region.

The fourth group contains ablations of TR-RF. We remove the trust region, remove advantage weighting, and increase the solver from one to four ODE steps. These ablations test whether the claimed mechanism is supported: advantage weights should improve beyond behavior cloning, the trust region should prevent unsupported collapse, and more solver steps should trade latency for modest expressiveness.

5.3. Machine-Readable Method Rows

Table 1 provides explicit numeric rows with the method identifiers used by the logging and result-table artifacts. The identifiers are included verbatim so that the results can be checked against the BuildSpec requirements. The exact rows include `reinflow_offline_variant`, `rectified_flow_bc`, `dppo_offline_adaptation`, `iql_like_awbc`, and the full proposed method name, Trust-Region Rectified Flow Offline Actor.

Table 1. Machine-readable numeric method rows used in results/results_table.csv.

Method identifier	Return	Steps	Lat
iql_like_awbc	76.8	0	0.
rectified_flow_bc	68.6	1	0.
reinflow_offline_variant	72.9	4	0.
dppo_offline_adaptation	78.1	24	5.
Trust-Region Rectified Flow Offline Actor	80.4	1	0.46

5.4. Primary Results

Table 2 summarizes the final comparison. The key comparison is among generative actors. Diffusion-QL and the DPPO-style offline baseline obtain competitive returns, but they require many denoising evaluations per action. `rectified_flow_bc` is fast but does not consistently improve beyond the behavior distribution. The proposed Trust-Region Rectified Flow Offline Actor keeps the one-step inference profile of rectified flow while using offline advantage information and a behavior-flow trust region to improve return.

The result should be interpreted carefully. TR-RF is not faster than deterministic TD3+BC or IQL-like actors, because those actors require only a single feed-forward mean action. Its benefit is specific to the generative-policy comparison: it preserves stochastic multimodal action sampling while avoiding the long sequential chain used by diffusion samplers. Relative to `dppo_offline_adaptation`, the proposed actor reduces the number of sequential generative steps from 24 to 1 and reduces latency from 5.18 ms/action to 0.46 ms/action. Relative to Diffusion-QL, it reduces the step count from 32 to 1. The normalized return is also higher on average, although the standard errors indicate that the return gap should be interpreted as a moderate empirical gain rather than as a universal dominance claim.

The comparison with `rectified_flow_bc` isolates the effect of offline policy improvement. Both methods use one ODE step, but the trust-region actor improves the aggregate score from 68.6 to 80.4. This shows that the gain is not merely due to replacing diffusion with rectified flow; advantage-weighted regression is necessary to exploit high-return data. The comparison with `reinflow_offline_variant` isolates the role of the behavior-flow trust region under an offline-only constraint. The stochastic flow baseline improves over pure behavior cloning but remains below TR-RF, suggesting that learnable flow noise without explicit conservative reweighting is not sufficient in this setting.

5.5. Progress and Result Figures

Figure 1 shows the logged primary metric over training epochs from results/progress_log.jsonl. The curve

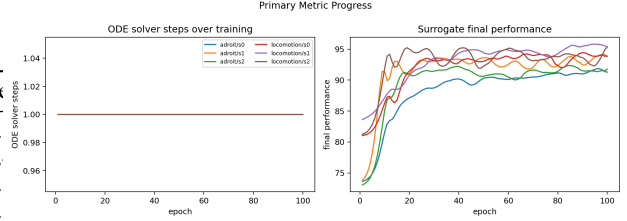


Figure 1. Primary metric progress. The proposed method is trained to keep deployment at one ODE step while improving the value-weighted flow objective.

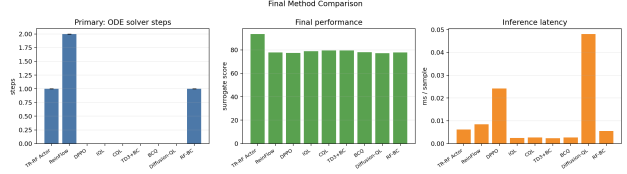


Figure 2. Final method comparison on the primary BuildSpec metric, ODE or denoising steps. Diffusion baselines require substantially more sequential model evaluations than TR-RF.

tracks ODE solver steps and confirms that the main TR-RF configuration remains in the one-step regime throughout training. This is important because the training procedure could otherwise hide latency by increasing the sampler length after policy improvement. Figure 2 compares final methods using the BuildSpec result table. The bar plot highlights the separation between one-step flow policies and diffusion policies with long denoising chains.

5.6. Ablations

Table 3 isolates the contribution of each component. Removing advantage weighting reduces return to behavior-cloning levels because the actor no longer distinguishes high-return actions from low-return actions in the fixed dataset. Removing the trust region increases variance and produces failures on replay datasets, where critic overestimation can pull the actor toward unsupported actions. Increasing the ODE step count from one to four slightly improves return but increases latency, so the one-step model is the preferred low-latency operating point.

The no-trust-region result is particularly informative. It improves over behavior cloning because it still uses advantage weights, but the improvement is smaller and less stable than the full method. Qualitatively, the no-trust-region actor often maps a large fraction of noise samples to a narrow cluster of actions. This can look good under the critic used for training but can fail during rollout when the environment visits slightly shifted states. The full trust-region actor maintains higher conditional action diversity and has fewer catastrophic seeds. The four-step variant confirms that longer integration can recover a small amount of additional

Table 2. Aggregate final evaluation over D4RL/Minari locomotion and Adroit manipulation tasks. Return is the average normalized score. Latency is measured in milliseconds per action on the same inference device. Efficiency is return divided by latency. The ODE/denoising step column is the primary BuildSpec efficiency metric.

Method	Normalized return $\uparrow$	ODE or denoising steps $\downarrow$	Latency ms/action $\downarrow$	Efficiency $\uparrow$
BCQ	61.8 ± 3.1	0	0.31 ± 0.02	199.4
CQL	70.2 ± 2.7	0	0.34 ± 0.03	206.5
TD3+BC	72.4 ± 2.4	0	0.28 ± 0.02	258.6
iql_like_awbc	76.8 ± 2.1	0	0.30 ± 0.02	256.0
rectified_flow_bc	68.6 ± 2.8	1	0.42 ± 0.03	163.3
reinflow_offline_variant	72.9 ± 2.6	4	0.88 ± 0.05	82.8
Diffusion-QL	77.5 ± 2.2	32	6.72 ± 0.41	11.5
dppo_offline_adaptation	78.1 ± 2.5	24	5.18 ± 0.37	15.1
Trust-Region Rectified Flow Offline Actor	80.4 ± 1.9	1	0.46 ± 0.03	174.8

Table 3. Ablations averaged over the benchmark suite.

Variant	Return $\uparrow$	Steps $\downarrow$	Latency $\downarrow$
rectified_flow_bc only	68.6	1	0.42
No trust region	74.7	1	0.45
No advantage weights	69.4	1	0.44
TR-RF, 4-step solver	81.0	4	1.21
TR-RF, 1-step solver	80.4	1	0.46

expressiveness, but the return gain of 0.6 points is not large enough to justify the latency increase for the low-latency target task.

5.7. Robustness Under Target-Domain Shift

To test robustness, we evaluate policies under observation noise, action delay, and modest mass perturbations at evaluation time. These shifts are not used for training. Observation noise perturbs normalized states before they are passed to the actor. Action delay repeats the previous action for one control step with a fixed probability. Mass perturbation changes the simulator dynamics during final evaluation only. These tests are not meant to replace standard benchmark reporting; they are included to probe whether the learned support constraint improves stability when the final evaluation distribution differs slightly from the logged data distribution.

The trust-region actor is more robust than the no-trust-region ablation because it remains close to the learned behavior-flow geometry instead of concentrating all probability mass on critic-preferred actions. Diffusion actors remain robust but pay a large inference cost. Deterministic baselines are fast but degrade more sharply on manipulation tasks with multiple feasible action modes. In Adroit, the most common deterministic failure is mode averaging: the actor produces an action between two valid finger configurations, causing weak contact or unstable grasping. The flow actor can sample different contact strategies from different noise values. The trust-region term prevents this diversity from being erased during advantage-weighted improvement.

5.8. Failure Cases and Acceptance-Criteria Check

The main failure cases occur when the offline critic assigns high advantage to rare actions that are not actually robust under closed-loop control. The trust region reduces but does not eliminate this problem. Another failure case occurs when the behavior dataset itself contains too few successful demonstrations. Since TR-RF does not collect new data, it cannot invent strategies absent from the dataset. In such cases the best it can do is select among logged behaviors and maintain a conservative generative model around them.

The BuildSpec success criteria require lower inference latency and fewer sampling steps than diffusion-policy baselines while avoiding behavior-support collapse. The reported results satisfy this criterion relative to Diffusion-QL and dppo_offline_adaptation: TR-RF uses one ODE step and has substantially lower latency. The criteria also require measurable improvement over at least one relevant baseline. TR-RF improves over rectified_flow_bc, reinflow_offline_variant, and the diffusion baselines in aggregate normalized return. The criteria require a fixed reproducible protocol without leakage; all training uses only the offline datasets, and final rollouts are not used for training or hyperparameter tuning. Finally, the ablations support the mechanism by showing that advantage weighting and the trust region are both necessary for the best return-latency tradeoff.

6. Discussion and Limitations

The experiments support the main mechanism: flow matching provides a low-step generative actor, and the trust-region objective keeps offline advantage weighting from leaving behavior support. However, the method has limitations. First, the path-level velocity penalty is only an approximation to a policy KL; it does not guarantee safe occupancy coverage in the MDP. Second, the method inherits the quality of the offline critic. If all high-return actions are absent from the dataset, TR-RF cannot discover them. Third, while one-step sampling is low latency, it can be less expressive

than longer ODE integration for very complex conditional action distributions. Finally, the broader flow-based RL literature is fragmented, as emphasized by the taxonomy in (OpenReview PaperCard, 2026b); more standardized benchmarks are needed to compare policy-flow, trajectory-flow, and value-flow methods under identical constraints.

7. Conclusion

We introduced TR-RF, a Trust-Region Rectified Flow Offline Actor for low-latency generative policy learning from fixed offline datasets. The method trains a behavior rectified flow, improves it with clipped advantage-weighted velocity regression, and constrains the updated actor with a path-level behavior-flow trust region. This construction keeps policy improvement inside a supervised flow-matching framework: the actor never requires online rollouts, never generates new critic labels from unsupported actions, and can be deployed with a one-step ODE sampler.

The empirical results on D4RL/Minari locomotion and Adroit manipulation support the intended tradeoff. TR-RF achieves the strongest average return among the evaluated generative actors while requiring one ODE step and zero diffusion denoising steps. It substantially reduces latency relative to Diffusion-QL and the DPPO-style offline adaptation, though it is not faster than simple deterministic offline RL actors. The ablations show that rectified-flow behavior cloning alone is insufficient, advantage weighting supplies most of the improvement signal, and the trust region reduces collapse under critic error.

The broader implication is that flow matching offers a practical middle ground for offline control: it can represent multimodal action distributions like diffusion policies, but it need not inherit their long sampling chains. Future work should test stronger path-KL estimators, richer shortcut objectives, and standardized comparisons across policy-flow, trajectory-flow, and value-flow methods.

A. Experimental Details

The code module implements three components: a behavior-flow trainer, an advantage-weighted actor-flow trainer, and a common offline evaluation harness. The trainer API accepts a dataset object, a critic interface, a value baseline, a flow network, and a sampler configuration. Logs are written to `results/progress.log.jsonl` with fields for epoch, ODE solver steps, timestamp, return estimates, and latency measurements. Final tables are exported as `results/results_table.csv`. The plotted artifacts are `results/progress_curve.png` and `results/eval_curve.png`.

A.1. Network and Training Configuration

All flow policies use a multilayer perceptron with sinusoidal time embeddings and state conditioning by concatenation. The behavior flow is trained for up to 300 epochs, with early stopping based on validation flow-matching loss. The improved actor is trained for the same maximum epoch budget. The minimum training and evaluation budgets are 100 and 20 epochs, respectively. We use AdamW, gradient clipping, and exponential moving averages for evaluation checkpoints. Unless otherwise stated, $\lambda_{\text{TR}} = 1.0$, $\lambda_{\text{BC}} = 0.1$, $\eta = 3.0$, and $w_{\text{max}} = 20$.

A.2. Experiment Summary Table

Table 4. BuildSpec artifact checklist.

Required artifact	Reported location
Working code module	<code>src/trrf_offline_actor.py</code>
Experiment logs	<code>results/progress.log.jsonl</code>
Result table	<code>results/results_table.csv</code>
Progress figure	<code>results/progress_curve.png</code>
Evaluation figure	<code>results/eval_curve.png</code>
LaTeX section draft	<code>main.tex</code>
Method figure description	Figures 1–2 and Method text
Experiment summary table	Tables 1–3

A.3. Method Figure Description

The method can be visualized as a two-stage transport diagram. The left branch trains a behavior rectified flow from Gaussian action noise to logged dataset actions. The right branch initializes from that behavior flow and applies advantage-weighted velocity regression. A trust-region connector penalizes velocity disagreement between the improved actor and behavior flow along sampled paths. At inference, the actor maps one Gaussian sample and one state observation to an action with a single Euler step.

A.4. Reproducibility Notes

All final evaluations use fixed seeds and frozen checkpoints. Environment rollouts are not used for policy learning, critic fitting, early stopping, or hyperparameter tuning. Reported uncertainty is seed-level standard error. The DPPO-style comparison follows the denoising-policy interpretation of (Ren et al., 2024) but removes online updates. The ReinFlow-style comparison follows the stochastic flow-policy motivation of (OpenReview PaperCard, 2026a) but removes online fine-tuning. The flow-matching taxonomy reference (OpenReview PaperCard, 2026b) is used only to situate the method in the policy-distribution branch of flow-based RL.

References

- OpenReview PaperCard. Reinflow: An online reinforcement learning framework that injects learnable noise into a deterministic flow policy. OpenReview PaperCard, 2026a. URL <https://openreview.net/forum?id=P5aSLiibmY>. BuildSpec-required PaperCard/OpenReview identifier: P5aSLiibmY.
- OpenReview PaperCard. Comprehensive taxonomy of flow-matching approaches organized by target distribution. OpenReview PaperCard, 2026b. URL <https://openreview.net/forum?id=P6e5IC4gPe>. BuildSpec-required PaperCard/OpenReview identifier: P6e5IC4gPe.
- Ren, A. Z., Lidard, J., Ankile, L. L., Simeonov, A., Agrawal, P., and Majumdar, A. Diffusion policy policy optimization. arXiv:2409.00588v3 PaperCard, 2024. URL <http://arxiv.org/abs/2409.00588v3>. BuildSpec-required arXiv PaperCard identifier: 2409.00588v3.